

Relatório Técnico: Sistema de Gestão para a Escolinha Raio de Sol

Gabriel Oliveira de Freitas

¹Instituto de Informática – Universidade Federal do Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

²Department of Computer Science – University of Durham
Durham, U.K.

³Departamento de Sistemas e Computação
Universidade Regional de Blumenau (FURB) – Blumenau, SC – Brazil

{nedel,flavio}@inf.ufrgs.br, R.Bordini@durham.ac.uk, jomi@inf.furb.br

Abstract. *This report details the development process of a management system for Escolinha Raio de Sol, an institution that serves elementary school students from first to fifth grade. A data management system was needed to better organize the school. The breakthrough proposal was a Java application structured using the Model-View-Controller (MVC) architectural pattern, capable of fully managing students, teachers, guardians, and classes. The system supports all CRUD (Create, Read, Update, Delete) operations and also features advanced features, such as student approval between school years and business rule validations to ensure data integrity. The project was successfully completed, resulting in a complete, documented, and tested system.*

Resumo. *Este relatório detalha o processo de desenvolvimento de um sistema de gestão para a Escolinha Raio de Sol, instituição que atende do primeiro ao quinto ano do ensino fundamental. Havia a necessidade de um sistema de gerenciamento de dados para uma melhor organização da escola. A solução proposta e implementada foi uma aplicação em Java, estruturada no padrão arquitetural Model-View-Controller (MVC), capaz de realizar o gerenciamento completo de Alunos, Professores, Respostas e Turmas. O sistema suporta todas as operações de CRUD (Create, Read, Update, Delete) e também possui funcionalidades avançadas, como a aprovação de alunos entre anos letivos e validações de regras de negócio para garantir a integridade dos dados. O projeto foi concluído com sucesso, resultando em um sistema completo, documentado e testado.*

1. Introdução

Em um cenário educacional cada vez mais dinâmico, a gestão eficiente de informações é um pilar para a qualidade do ensino e a organização administrativa. Instituições de ensino lidam com uma grande quantidade de dados, como cadastros de alunos, professores, históricos de turmas e o vínculo com os responsáveis. Na Escolinha Raio de Sol, esses processos eram realizados de forma manual, resultando em um fluxo de trabalho lento e

com maior margem de erro. A necessidade de consultar o histórico de um aluno ou verificar a disponibilidade de um professor, por exemplo, demandava muito mais trabalho, consumindo tempo e recursos.

Diante deste contexto, o problema se define como a necessidade de um sistema de dados e informações para a escola. A solução proposta neste projeto foi o desenvolvimento de uma aplicação em Java, que serve como uma ferramenta de administração completa. O sistema foi projetado para automatizar as operações de cadastro, consulta, atualização e exclusão (CRUD) de todas as principais entidades da escola: Alunos, Professores, Responsáveis e Turmas. A utilização da linguagem Java e da Programação Orientada a Objetos permitiu a criação de um modelo de dados coeso e representativo da realidade da escola, resultando em um sistema de alta qualidade e fácil manutenção.

2. Fundamentação Teórica

A construção de uma aplicação de software robusta e sustentável requer a aplicação de conceitos teóricos consolidados. Este projeto foi fundamentado em três pilares principais: a linguagem de programação Java, os paradigmas da Programação Orientada a Objetos (POO) e o padrão de arquitetura Model-View-Controller (MVC).

2.1. Linguagem de Programação Java

A construção de uma aplicação de software completa e sustentável exige a aplicação de alguns conceitos teóricos consolidados. Este projeto foi fundamentado em três pilares principais: a linguagem de programação Java, a Programação Orientada a Objetos (POO) e o padrão de arquitetura Model-View-Controller (MVC).

2.2. Programação Orientada a Objetos (POO)

A POO foi essencial para modelar a complexidade do sistema da escola. Os conceitos fundamentais foram aplicados da seguinte forma:

- **Encapsulamento:** Os atributos de todas as classes de modelo (ex: nome, cpf na classe Pessoa) foram declarados como private. O acesso a esses dados é controlado exclusivamente por métodos públicos get e set, protegendo a integridade do estado interno dos objetos.
- **Herança:** Para fazer o reuso de código, foi criada uma superclasse Pessoa que abstrai os atributos comuns a alunos, professores e responsáveis. As classes Aluno, Professor e Responsavel herdam de Pessoa, além de ter seus próprios atributos e comportamentos.
- **Polimorfismo:** O polimorfismo é aplicado, por exemplo, na gestão de responsáveis. Um Aluno possui um atributo do tipo Pessoa como seu responsável. Isso permite que tanto um objeto Professor quanto um Responsavel possam assumir esse papel, sendo tratados da mesma forma pelo sistema, que se adapta ao tipo específico do objeto quando necessário.

2.3. Padrão de Projeto MVC (Model-View-Controller)

A arquitetura do sistema foi o padrão MVC para garantir a separação de responsabilidades, um requisito chave do projeto.

- **Model:** Representa os dados e as regras de negócio. No projeto, corresponde ao pacote `model`, contendo classes como `Pessoa`, `Aluno`, `Professor`, `Responsavel`, `Endereco` e `Turma`.
- **View:** É a camada de apresentação, responsável pela interface com o usuário. Corresponde ao pacote `view` (classe `Main`), que exibe menus e recebe entradas via console.
- **Controller:** Atua como intermediário, recebendo solicitações da View e manipulando o Model para atendê-las. Corresponde ao pacote `controller`, que organiza toda a lógica da aplicação. Possui as classes: `EscolaController`, `AlunoController`, `ProfessorController`, `ResponsavelController` e `TurmaController`.

Essa separação torna o sistema mais flexível a mudanças. Uma alteração na interface do usuário (View) não afeta a lógica de negócio (Controller), e vice-versa.

3. Metodologia

3.1. Requisitos e Funcionalidades

Com base no documento do problema, foram definidos os seguintes requisitos funcionais:

- **CRUD de Alunos:** Permitir o cadastro, busca, listagem, atualização e exclusão de alunos.
- **CRUD de Professores:** Permitir o cadastro, busca, listagem, atualização e exclusão de professores.
- **CRUD de Responsáveis:** Permitir o cadastro, busca, listagem, atualização e exclusão de responsáveis.
- **CRUD de Turmas:** Permitir a criação, busca, listagem, atualização e exclusão de turmas.
- **Funcionalidades Extras:** Fazer a matrícula de alunos em turmas, a associação de responsáveis a alunos e a aprovação de alunos entre anos letivos.

3.2. Processo de Codificação e Escolhas Técnicas

O desenvolvimento foi dividido em etapas. Primeiramente, foi construída a camada Model, definindo todas as classes e seus relacionamentos. Em seguida, foi desenvolvida a camada Controller, começando pelos controllers individuais (`AlunoController`, etc.) e finalizando com o `EscolaController`, para simplificar a comunicação com a View. A View (classe `Main`) foi a última camada a ser implementada.

Uma escolha técnica importante foi o uso de `ArrayList` para armazenar as listas de objetos em memória. Para a entrada de dados do usuário, a abordagem inicial com `scanner.nextInt()` foi substituída por `Integer.parseInt(scanner.nextLine())` dentro de um bloco try-catch, uma solução mais completa que evita `InputMismatchException` e problemas de buffer do scanner.

3.3. Ficha Técnica

Sistema Operacional: Windows 11

Linguagem: Java (JDK 24)

IDE: IntelliJ

Framework de Testes: JUnit 5

4. Resultados e Discussões

4.1. Manual de Uso

Ao iniciar o programa, o usuário se depara com um menu principal que o direciona para o gerenciamento de alunos, professor, responsável e turma. O uso do sistema é intuitivo:

1. **Pré-cadastros:** Antes de cadastrar um aluno, o usuário deve primeiro cadastrar seu responsável (seja um Responsavel ou um Professor). Antes de criar uma turma, o Professor regente já deve existir no sistema.
2. **Operações de CRUD:** Dentro de cada gerenciador, o usuário pode escolher as opções de CRUD. O sistema guia o usuário em cada etapa, solicitando os dados necessários.
3. **Avanço de Ano Letivo:** Para usar esta função, o usuário deve primeiro cadastrar manualmente as turmas do ano de destino. Em seguida, ao selecionar a opção, ele informa o ano de origem e o de destino, e o sistema promove os alunos automaticamente.

4.2. Dados de Entrada e Saída

Dados de Entrada: Todas as entradas são fornecidas pelo usuário via texto no console. Exemplo: CPFs ("123.456.789-00"), nomes ("Gabriel Oliveira"), opções de menu ("1"), anos letivos ("2025"), etc. O sistema realiza validações para garantir a integridade desses dados.

Dados de Saída: As saídas são mensagens de texto formatadas, exibidas no console. Elas incluem: confirmações de sucesso ("Aluno cadastrado com sucesso!"), mensagens de erro ("Este CPF já está cadastrado..."), e listagens de dados formatadas (ex: "Nome: Aluno Teste — CPF: 123.456.789-00").

4.3. Testes e Erros Encontrados

Os testes de unidade foram cruciais para garantir a qualidade. Utilizando o JUnit 5, foram criados testes para cada método público dos controllers, validando a lógica de negócio. Por exemplo, um teste verifica se a adição de um aluno com CPF duplicado é corretamente barrada, enquanto outro confirma se a função de avançar o ano letivo move o aluno para a turma correta. Durante o desenvolvimento, um bug foi identificado nos menus devido à ausência de "break;" em alguns case do switch, o que foi corrigido. Além disso nos primeiros testes de menu, ao digitar uma letra, havia uma "quebra no código", pelo fato de só dados do tipo inteiro serem aceitos. Esse problema foi solucionado através da implementação de um bloco try-catch para tratar a exceção NumberFormatException, utilizando o método Integer.parseInt() para validar a entrada do usuário.

5. Conclusão

O sistema desenvolvido atendeu plenamente a todos os requisitos propostos, entregando uma solução funcional para o gerenciamento da Escolinha Raio de Sol. Em termos de requisitos obrigatórios, todos os objetivos foram alcançados com sucesso. Foram implementadas todas as operações de:

CRUD (Create, Read, Update e Delete) para as quatro entidades principais: Alunos, Professores, Responsáveis e Turmas. A arquitetura do sistema seguiu estritamente o padrão

Model-View-Controller (MVC), garantindo a separação de responsabilidades e a manutenibilidade do código. Adicionalmente, todo o código-fonte foi documentado utilizando o padrão

Javadoc e a qualidade da lógica de negócio foi assegurada pela criação de **testes de unidade** com JUnit.

Além disso, há de se destacar a validação de unicidade de CPF em todo o sistema (impedindo que um aluno, professor e responsável compartilhem o mesmo CPF), a implementação de regras de negócio específicas para a alocação de professores em turmas, e o desenvolvimento de uma rotina semi-automática para avançar o ano letivo, que promove os alunos para suas séries próximas, otimizando o processo de administração da gestão escolar.

5.1. Trabalhos Futuros

Adições com potencial para agregar ao projeto e aprendizado podem ser a implementação de:

Persistência de Dados: Implementar a gravação dos dados em arquivos (ex: JSON) ou em um banco de dados para que as informações não se percam.

Interface Gráfica (GUI): Desenvolver uma interface gráfica com JavaFX ou Swing .

Sistema de Avaliações: Reintroduzir o Boletim com um sistema de notas, impactando a lógica de aprovação/reprovação no avanço do ano letivo.

6. Referências

DEITEL, P.; DEITEL, H. *Java: Como Programar*. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

JUNIT TEAM. JUnit 5 User Guide. Disponível em: <https://junit.org/junit5/docs/current/user-guide/>. Acesso em: 04 set. 2025.